

Lista: Introdução ao Prolog.

Nome: _____ Matrícula: _____

Exercícios baseados no capítulo 19 do livro Modern Programming Languages.

Exercício 01. Defina os predicados abaixo em termos das relações `parent(X, Y)`, `female(X)` e `male(X)`. Para testar seu código, defina alguns fatos de parentesco como os utilizados nas notas de aula, juntamente com fatos para `female` e `male`. Suas soluções devem ser gerais o suficiente para funcionar com qualquer conjunto desses fatos.

- a) Defina um predicado `mother` para que `mother(X, Y)` diga que `X` é a mãe de `Y`.
- b) Defina um predicado `father` para que `father(X, Y)` diga que `X` é o pai de `Y`.
- c) Defina um predicado `sister` para que `sister(X, Y)` diga que `X` é irmã de `Y`. Tenha cuidado, uma pessoa não pode ser sua própria irmã.
- d) Defina um predicado `grandson` para que `grandson(X, Y)` diga que `X` é neto de `Y`.
- e) Defina um predicado `firstCousin` para que `firstCousin(X, Y)` diga que `X` é primo de primeiro grau de `Y`. Tenha cuidado, uma pessoa não pode ser seu próprio primo, nem um irmão ou irmã pode ser primo.
- f) Defina o predicado `descendant` para que `descendant(X, Y)` diga que `X` é um descendente de `Y`.

Exercício 02. Defina um predicado `third` para que `third(X, Y)` diga que `Y` é o terceiro elemento da lista `X`. (O predicado deve falhar se `X` tiver menos de três elementos.) Dica: Isto pode ser expresso como um fato.

Exercício 03. Defina um predicado `firstPair` para que `firstPair(X)` tenha sucesso se e somente se `X` for uma lista de pelo menos dois elementos, com o primeiro elemento igual ao segundo. Dica: Isto pode ser expresso como um fato.

Exercício 04. Defina um predicado `del3` para que `del3(X, Y)` diga que a lista Y é igual à lista X , mas com o terceiro elemento excluído. (O predicado deve falhar se X tiver menos de três elementos.) Dica: Isto pode ser expresso como um fato.

Exercício 05. Defina um predicado `dupList` para que `dupList(X, Y)` diga que X é uma lista e Y é a mesma lista, mas com cada elemento de X repetido duas vezes consecutivas. Por exemplo, se X é $[1, 3, 2]$, Y deve ser $[1, 1, 3, 3, 2, 2]$. Se X é $[]$, Y deve ser $[]$. Verifique se o seu predicado funciona em ambas as direções — ou seja, verifique se ele funciona em consultas como `dupList(X, [1, 1, 3, 3, 2, 2])` e `dupList([1, 3, 2], Y)`.

Exercício 06. Defina o predicado `oddSize` para que `oddSize(X)` diga que X é uma lista cujo comprimento é um número ímpar. Dica: Você não precisa calcular o comprimento exato, nem fazer qualquer aritmética inteira.

Exercício 07. Defina o predicado `evenSize` para que `evenSize(X)` diga que X é uma lista cujo comprimento é um número par. Dica: Você não precisa calcular o comprimento exato, nem fazer qualquer aritmética inteira.